PYTHON PROGRAMMING

Assignment 2 – CO2 Report

Control flow, string operations, functions and recursion · 50 Marks · Individual · Submission date: 30/10/2026

Item	Details
Name	Dhaarani D
Register Number	_____
GitHub repository link	_____
Python version	3.12.3 (standard library only)

Contents

Question	Topic	Marks	Bloom's level
Q1	Decisions with many boundaries (if / elif / else)	6	K4
Q2	Loops that count attempts (while, for, break, continue, pass, loop-else)	4	K3
Q3	Vehicle registration number validator (string operations, slicing)	6	K4
Q4	Building long text: + or join()	6	K4
Q5	Functions, parameters and scope	7	K4
Q6	Recursion and its limits	8	K5
Q7	Day-4 design challenge: diagnose, redesign and validate	13	K5
	Total	**50**	

PYTHON PROGRAMMING · ASSIGNMENT 2 (CO2)

Q1 – Decisions with Many Boundaries

6 Marks · Bloom's Level: K4 – Analyse · Individual assignment · Submission date: 30/10/2026

Name: Dhaarani D	**Register Number:** _____
GitHub repository: _____	**Repository folder:** Task01/

1. Problem and rule

A household receives a subsidy according to its monthly consumption. The rule, restated as a table:

Units consumed	Condition	Decision
0 – 100	none	FULL subsidy
101 – 200	household holds a ration card	PARTIAL subsidy (otherwise NONE)
above 200	household has a member above 80 years	SENIOR subsidy (otherwise NONE)

The developer wrote the rule with several separate `if` statements, and some households received two subsidies. The buggy version used in this report is:

Developer's version (separate if statements)

```
if units <= 100: result.append('FULL')
if units <= 200 and ration: result.append('PARTIAL')   # also true when units <= 100
if units > 100 and senior: result.append('SENIOR')     # also true when 101 <= units <= 200
```

2. Predictions made before running any code

Eight households were chosen so that every boundary (100, 101, 200, 201) is tested, with and without a ration card and a senior member. The expected decision is worked out from the rule; the last column is my prediction of what the **separate-if** version would return, by tracing each `if` by hand.

#	Units	Ration card	Member > 80	Predicted decision (rule)	Reasoning	Predicted separate-if result
1	50	No	No	FULL	units ≤ 100	[FULL]
2	100	Yes	No	FULL	boundary: 100 is still in the first slab	[FULL, PARTIAL] – 2nd if is also true
3	101	Yes	No	PARTIAL	boundary: first unit of the 2nd slab, has card	[PARTIAL]
4	101	No	No	NONE	2nd slab but no ration card	[ ] – nothing
5	200	Yes	Yes	PARTIAL	boundary: 200 is still 2nd slab; senior does not matter here	[PARTIAL, SENIOR] – 3rd if (units > 100) is true
6	200	No	Yes	NONE	senior rule applies only above 200	[SENIOR] – wrongly given
7	201	No	Yes	SENIOR	boundary: first unit of the 3rd slab, has senior member	[SENIOR]
8	201	Yes	No	NONE	ration card does not help above 200	[ ] – nothing

3. Why separate `if` statements can give more than one result

Every `if` is a complete, independent test. Python evaluates each condition one after another, and **each body whose condition is true is executed**. If the conditions overlap, several bodies run. For 100 units with a ration card, `units <= 100` and `units <= 200 and ration` are both true, so FULL and PARTIAL are both added (row 2). For 200 units with a ration card and a senior member, the second and third tests are both true (row 5).

In an `if–elif–else` chain the conditions form **one decision**: the first true condition runs its body and **all remaining `elif`/`else` parts are skipped**. At most one branch runs, and because the final `else` has no condition, exactly one branch always runs. The overlap still exists in the data, but it can no longer produce two results; the order of the branches decides who wins. Chained comparisons such as `101 <=`

`units <= 200` also make each slab's two ends visible in one expression.

4. Implementations (source code)

Task01/subsidy.py – separate ifs, if–elif–else, table lookup, match–case, tests and timing

```
"""Q1 - State electricity subsidy: separate ifs vs if-elif-else, plus alternatives."""
import time

# ---------- 1. Implementation with SEPARATE if statements (the developer's buggy version) ----------
def subsidy_separate(units, ration, senior):
    result = []
    if units <= 100:
        result.append("FULL")
    if units <= 200 and ration:            # bug: also true when units <= 100
        result.append("PARTIAL")
    if units > 100 and senior:             # bug: should be units > 200
        result.append("SENIOR")
    return result

# ---------- 2. Implementation with if-elif-else (exactly one outcome) ----------
def subsidy_elif(units, ration, senior):
    if units <= 100:
        return "FULL"
    elif 101 <= units <= 200 and ration:   # chained comparison
        return "PARTIAL"
    elif units > 200 and senior:
        return "SENIOR"
    else:
        return "NONE"

# ---------- 3. Alternative: table of (upper limit, rule) searched in a loop ----------
SLABS = [
    (100,           lambda ration, senior: "FULL"),
    (200,           lambda ration, senior: "PARTIAL" if ration else "NONE"),
    (float("inf"),  lambda ration, senior: "SENIOR" if senior else "NONE"),
]
def subsidy_table(units, ration, senior):
    for upper, rule in SLABS:
        if units <= upper:
            return rule(ration, senior)

# ---------- 4. Alternative: match-case (Python 3.10+) ----------
def subsidy_match(units, ration, senior):
    match (units, ration, senior):
        case (u, _, _) if u <= 100:
            return "FULL"
        case (u, True, _) if u <= 200:
            return "PARTIAL"
        case (u, False, _) if u <= 200:
            return "NONE"
        case (_, _, True):
            return "SENIOR"
        case _:
            return "NONE"

# ---------- Predictions written BEFORE running (units, ration card, member above 80) ----------
households = [
    (50,  False, False, "FULL"),
    (100, True,  False, "FULL"),
    (101, True,  False, "PARTIAL"),
    (101, False, False, "NONE"),
    (200, True,  True,  "PARTIAL"),
    (200, False, True,  "NONE"),
    (201, False, True,  "SENIOR"),
    (201, True,  False, "NONE"),
]

print(f"{'units':>5} {'ration':>6} {'senior':>6} | {'predicted':9} | {'separate ifs':22} | {'if-elif-else':12} |
    ok")
for u, r, s, pred in households:
    sep = subsidy_separate(u, r, s)
    one = subsidy_elif(u, r, s)
    ok = "yes" if one == pred else "NO"
    flag = "  <-- TWO subsidies" if len(sep) > 1 else ""
    print(f"{u:>5} {str(r):>6} {str(s):>6} | {pred:9} | {str(sep):22} | {one:12} | {ok}{flag}")

# ---------- Equivalence check of all implementations over a big input space ----------
mismatch = 0
for u in range(0, 400):
    for r in (False, True):
        for s in (False, True):
            a = subsidy_elif(u, r, s)
            if not (a == subsidy_table(u, r, s) == subsidy_match(u, r, s)):
                mismatch += 1
```

```
print("\nelif / table / match agree on all 1600 inputs:", mismatch == 0)
buggy = sum(1 for u in range(0, 400) for r in (False, True) for s in (False, True)
            if len(subsidy_separate(u, r, s)) > 1)
print("inputs where the separate-if version gives MORE THAN ONE subsidy:", buggy)

# ---------- Timing per decision ----------
def bench(fn, n=200_000):
    cases = [(50, False, False), (150, True, False), (500, False, True)]
    t = time.perf_counter()
    for _ in range(n // 3):
        for c in cases:
            fn(*c)
    return (time.perf_counter() - t) / n * 1e9
print("\nns per decision (3 slabs, avg of 3 cases):")
for name, fn in [("if-elif-else", subsidy_elif), ("table loop", subsidy_table), ("match-case", subsidy_match)]:
    print(f"  {name:13} {bench(fn):7.1f} ns")
```

5. Observed results

Program output

```
units ration senior | predicted | separate ifs           | if-elif-else | ok
   50  False  False | FULL      | ['FULL']               | FULL         | yes
  100   True  False | FULL      | ['FULL', 'PARTIAL']    | FULL         | yes  <-- TWO subsidies
  101   True  False | PARTIAL   | ['PARTIAL']            | PARTIAL      | yes
  101  False  False | NONE      | []                     | NONE         | yes
  200   True   True | PARTIAL   | ['PARTIAL', 'SENIOR']  | PARTIAL      | yes  <-- TWO subsidies
  200  False   True | NONE      | ['SENIOR']             | NONE         | yes
  201  False   True | SENIOR    | ['SENIOR']             | SENIOR       | yes
  201   True  False | NONE      | []                     | NONE         | yes

elif / table / match agree on all 1600 inputs: True
inputs where the separate-if version gives MORE THAN ONE subsidy: 302

ns per decision (3 slabs, avg of 3 cases):
  if-elif-else     54.9 ns
  table loop      118.3 ns
  match-case      110.0 ns
```

Comparison at every boundary. All eight observed if–elif–else decisions equal my predictions (column `ok` = yes). The separate-if version returned two subsidies at 100 units with a ration card and at 200 units with a ration card and a senior member, and returned SENIOR at 200 units without a ration card, which is wrong because the senior rule starts only above 200. Over all 1,600 inputs tested (units 0–399 × ration card × senior member), the separate-if version gave more than one subsidy for **302** inputs; the if–elif–else version never did.

Boundary	Rule says	if-elif-else	separate ifs (ration card & senior member)
100 units	FULL	FULL	FULL + PARTIAL (card)
101 units	PARTIAL / NONE	PARTIAL / NONE	correct here
200 units	PARTIAL / NONE	PARTIAL / NONE	PARTIAL + SENIOR, or SENIOR alone: wrong
201 units	SENIOR / NONE	SENIOR / NONE	correct here

6. Design decision: nested `if` or combined Boolean conditions?

A nested version would look like this:

Nested version (for comparison)

```
if units <= 100:
    return 'FULL'
else:
    if units <= 200:
        if ration:
            return 'PARTIAL'
        else:
            return 'NONE'
    else:
        if senior:
            return 'SENIOR'
        else:
            return 'NONE'
```

My decision: a flat if–elif–else chain with combined Boolean conditions (`101 <= units <= 200 and ration`). Reasons:

- **Readability:** each line is one self-contained business rule – slab and extra condition together – so a reviewer can compare the code with the policy line by line. In the nested version the reader must keep three levels of indentation in mind to know which `else` belongs to which `if`.
- **Ease of change:** if the government changes the ration-card limit from 200 to 250, only one number on one line changes. If a new slab is added, one `elif` is inserted. In the nested version both the outer and the inner structure must be edited and a mis-matched `else` is easy to create.
- **Fewer duplicated outcomes:** the nested version returns 'NONE' in two places; the flat chain has a single final `else`.
- **Where nesting would win:** if many slabs shared the same extra condition (for example every slab needed a ration card), nesting once on that condition would avoid repeating `and ration` – that is not the case in this rule, so the flat chain is better here.

7. Alternative concept and complexity analysis

Both alternatives are implemented in `subsidy.py` and produce identical decisions to if–elif–else on all 1,600 test inputs (`agree ... True`).

Table (list of (upper limit, decision) pairs) searched in a loop

```
SLABS = [(100, rule_full), (200, rule_partial), (inf, rule_senior)]
for upper, rule in SLABS:
    if units <= upper:
        return rule(ration, senior)
```

match–case (Python 3.10+)

```
match (units, ration, senior):
    case (u, _, _) if u <= 100:      return 'FULL'
    case (u, True, _) if u <= 200:   return 'PARTIAL'
    case (u, False, _) if u <= 200:  return 'NONE'
    case (_, _, True):               return 'SENIOR'
    case _:                          return 'NONE'
```

Approach	Time per decision for k slabs	Space	Measured (k = 3)	Remarks
if–elif–else	O(k) worst case (tests tried in order); O(1) for a fixed small k	O(1)	54.9 ns	fastest here; the rule is code, so a change means editing code
Table loop	O(k) linear search (O(log k) if the limits are sorted and `bisect` is used)	O(k) for the table	118.3 ns	new slab = one new row; data-driven, can be loaded from a file
match–case	O(k): cases are tried top to bottom	O(1)	110.0 ns	pattern syntax is clear for tuples, but needs `if` guards for ranges, so it has no real advantage over elif here

With only k = 3 slabs, the constant costs (function call of the rule, tuple creation) dominate and all three take about 50–120 nanoseconds, so the difference is irrelevant for a billing system. The table approach becomes better as the number of slabs grows (for example 50 tariff slabs with binary search) or when the slabs must change without editing code. Timings were measured with `time.perf_counter()` on CPython 3.12.3 and will differ on another computer.

8. Conclusion

Overlapping conditions in separate `if` statements gave two subsidies for 302 of 1,600 inputs, including at the 100 and 200 boundaries. A flat `if–elif–else` chain with chained comparisons and combined Boolean conditions gives exactly one decision, matches all eight predictions and is the easiest structure to read and change.

PYTHON PROGRAMMING · ASSIGNMENT 2 (CO2)

Q2 – Loops that Count Attempts

4 Marks · Bloom's Level: K3 – Apply · Individual assignment · Submission date: 30/10/2026

Name: Dhaarani D	**Register Number:** _____
GitHub repository: _____	**Repository folder:** Task02/

1. Problem

A bank kiosk sends an OTP and allows **three attempts**. After three wrong attempts the account is locked for 30 minutes. A **blank entry must not count** as an attempt. The correct OTP used for testing is `482913` and the keypad is simulated by feeding a list of typed entries.

2. Design A – `while` loop

The loop condition `attempts < max_attempts` is checked before every pass. The counter `attempts` is increased **only after** the blank check, so blank entries are skipped by `continue` without being counted.

Design A (extract of Task02/otp.py)

```
def verify_while(read, max_attempts=3):
    attempts = 0
    while attempts < max_attempts:
        entry = read().strip()
        if entry == "":
            continue                    # blank: NOT counted, ask again
        attempts += 1                   # counter updated only for real entries
        if entry == OTP:
            return "ACCESS GRANTED", attempts
        else:
            pass                        # placeholder: alert for a wrong try to be added later
    else:                               # runs only when the loop ended without break/return
        return "ACCOUNT LOCKED 30 min", attempts
```

3. Design B – `for` loop with `range(3)`

`range(3)` already limits the loop to three passes, so no counter is needed. The difficulty is the blank entry: if `continue` were used on a blank, that pass of the `for` loop would already be used up. A small inner `while True` loop therefore keeps asking until a non-blank entry arrives, and `break` leaves the inner loop.

Design B (extract of Task02/otp.py)

```
def verify_for(read):
    for attempt in range(1, 4):
        while True:                     # inner loop only to skip blanks
            entry = read().strip()
            if entry != "":
                break                   # real entry: leave the inner loop
        if entry == OTP:
            return "ACCESS GRANTED", attempt
    else:
        return "ACCOUNT LOCKED 30 min", 3
```

4. Where `break`, `continue`, `pass` and loop–`else` are needed

Statement	Where it is used	Why it is needed
`continue`	Design A: blank entry	jumps to the next pass of the loop **before** the counter is increased, so a blank is not an attempt
`break`	Design B inner loop; Design A2 (`verify_while_break`) when the OTP is correct	leaves the loop immediately; in A2 it also makes Python **skip** the `else` clause
`pass`	wrong-OTP branch	an empty branch that must exist syntactically; marks where an SMS alert to the customer can later be added
loop–`else`	after the loop in A, A2, B	runs only when the loop finished **without** `break`, that is, all three real attempts failed, so this is exactly the place to lock the account

In Design A the correct OTP leaves the function by `return`, which also skips the `else`. Variant A2, shown in the source code below, uses `break` and a flag so that the role of `break` with loop–`else` is visible:

Design A2 (extract)

```
    while attempts < max_attempts:
        ...
        if entry == OTP:
            granted = True
            break                          # the else clause below is skipped
    else:
        return "ACCOUNT LOCKED 30 min", attempts    # no break: three real failures
    return "ACCESS GRANTED", attempts
```

5. Source code and observed results

Task02/otp.py (complete program)

```
"""Q2 - OTP verification: while, for/range(3), break/continue/pass, loop-else, changed condition, recursion."""
import sys, time

OTP = "482913"

def make_input(entries):
    """Simulated keypad: returns the next typed entry each time it is called."""
    it = iter(entries)
    return lambda: next(it)

# ---------------- Design A: while loop ----------------
def verify_while(read, max_attempts=3):
    attempts = 0
    while attempts < max_attempts:
        entry = read().strip()
        if entry == "":
            continue                       # blank: NOT counted, ask again
        attempts += 1                      # counter updated only for real entries
        if entry == OTP:
            return "ACCESS GRANTED", attempts
        else:
            pass                           # placeholder: SMS alert for a wrong try to be added later
    else:                                  # runs only when the loop ended without break/return
        return "ACCOUNT LOCKED 30 min", attempts


# ---------------- Design A2: while loop that uses break (loop-else locks only if break never ran) ----------------
def verify_while_break(read, max_attempts=3):
    attempts, granted = 0, False
    while attempts < max_attempts:
        entry = read().strip()
        if entry == "":
            continue                       # blank: not counted
        attempts += 1
        if entry == OTP:
            granted = True
            break                          # leave the loop: the else clause below is skipped
        else:
            pass                           # placeholder for a future wrong-OTP alert
    else:
        return "ACCOUNT LOCKED 30 min", attempts      # no break happened: three real failures
    return "ACCESS GRANTED", attempts

# ---------------- Design B: for loop with range(3) ----------------
def verify_for(read):
    for attempt in range(1, 4):
        while True:                        # inner loop only to skip blanks
            entry = read().strip()
            if entry != "":
                break                      # real entry: leave the inner loop
        if entry == OTP:
            return "ACCESS GRANTED", attempt
    else:
        return "ACCOUNT LOCKED 30 min", 3

# ---------------- The classic bug: counter updated in the wrong place ----------------
def verify_while_buggy(read, max_attempts=3):
    attempts = 0
    while attempts < max_attempts:
        attempts += 1                      # BUG: counts blank entries too
        entry = read().strip()
        if entry == "":
            continue
        if entry == OTP:
            return "ACCESS GRANTED", attempts
    return "ACCOUNT LOCKED 30 min", attempts
```

```
# ---------------- Changed condition: two extra attempts after a correct security answer ----------------
def verify_extra(read, security_ok, max_attempts=3):
    attempts, allowed, bonus_used = 0, max_attempts, False
    while True:
        while attempts < allowed:
            entry = read().strip()
            if entry == "":
                continue
            attempts += 1
            if entry == OTP:
                return "ACCESS GRANTED", attempts
        if not bonus_used and security_ok():      # ask security question once
            allowed += 2
            bonus_used = True
        else:
            return "ACCOUNT LOCKED 30 min", attempts

# ---------------- Alternative concept: recursion ----------------
def verify_rec(read, attempt=1, max_attempts=3):
    if attempt > max_attempts:
        return "ACCOUNT LOCKED 30 min", max_attempts
    entry = read().strip()
    if entry == "":
        return verify_rec(read, attempt, max_attempts)      # blank: same attempt number
    if entry == OTP:
        return "ACCESS GRANTED", attempt
    return verify_rec(read, attempt + 1, max_attempts)

# ---------------- Demonstration ----------------
scenarios = {
    "right on 2nd try":             ["111111", "482913"],
    "blank, wrong, blank, wrong, right": ["", "000000", "  ", "123456", "482913"],
    "three wrong":                  ["1", "2", "3"],
    "blank, 3 wrong":               ["", "1", "", "2", "3"],
}
print(f"{'scenario':36} {'while':28} {'for':28} {'recursive':28}")
for name, e in scenarios.items():
    w = verify_while(make_input(e))
    f = verify_for(make_input(e))
    r = verify_rec(make_input(e))
    print(f"{name:36} {str(w):28} {str(f):28} {str(r):28}")

print("\nwhile+break variant agrees with the first while version on all scenarios:",
      all(verify_while_break(make_input(e)) == verify_while(make_input(e)) for e in scenarios.values()))

print("\nBug demo (blank counts as an attempt) on ['', '1', '2', '482913']:")
print("  correct while :", verify_while(make_input(["", "1", "2", "482913"])))
print("  buggy while   :", verify_while_buggy(make_input(["", "1", "2", "482913"])))

print("\nChanged condition (two extra attempts after correct security answer):")
e = ["1", "2", "3", "4", "482913"]
print("  security answer correct :", verify_extra(make_input(e), lambda: True))
print("  security answer wrong   :", verify_extra(make_input(e), lambda: False))
e = ["1", "2", "3", "4", "5", "6", "482913"]
print("  correct, but 5 wrong tries:", verify_extra(make_input(e), lambda: True))

# ---------------- Complexity: stack depth, time for a attempts ----------------
def loop_attempts(a):
    n = 0
    while n < a:
        n += 1
    return n
def rec_attempts(a, n=0):
    if n == a:
        return n
    return rec_attempts(a, n + 1)

print("\nrecursion limit:", sys.getrecursionlimit())
for a in (3, 100, 900, 5000):
    t = time.perf_counter(); loop_attempts(a); tl = time.perf_counter() - t
    try:
        t = time.perf_counter(); rec_attempts(a); tr = time.perf_counter() - t
        res = f"{tr*1e6:8.1f} us"
    except RecursionError:
        res = "RecursionError"
    print(f"  a={a:5}  loop {tl*1e6:8.1f} us   recursion {res}")
```

Program output

```
scenario                            while                          for                            recursive

right on 2nd try                    ('ACCESS GRANTED', 2)        ('ACCESS GRANTED', 2)        ('ACCESS GRANTED',
    2)
blank, wrong, blank, wrong, right    ('ACCESS GRANTED', 3)        ('ACCESS GRANTED', 3)        ('ACCESS GRANTED',
    3)
three wrong                         ('ACCOUNT LOCKED 30 min', 3) ('ACCOUNT LOCKED 30 min', 3) ('ACCOUNT LOCKED 30
```

```
     min', 3)
blank, 3 wrong                       ('ACCOUNT LOCKED 30 min', 3) ('ACCOUNT LOCKED 30 min', 3) ('ACCOUNT LOCKED 30
     min', 3)

while+break variant agrees with the first while version on all scenarios: True

Bug demo (blank counts as an attempt) on ['', '1', '2', '482913']:
  correct while : ('ACCESS GRANTED', 3)
  buggy while   : ('ACCOUNT LOCKED 30 min', 3)

Changed condition (two extra attempts after correct security answer):
  security answer correct : ('ACCESS GRANTED', 5)
  security answer wrong   : ('ACCOUNT LOCKED 30 min', 3)
  correct, but 5 wrong tries: ('ACCOUNT LOCKED 30 min', 5)

recursion limit: 1000
  a=    3  loop      0.9 us   recursion      1.0 us
  a=  100  loop      1.7 us   recursion      9.8 us
  a=  900  loop     15.2 us   recursion    128.7 us
  a= 5000  loop     88.1 us   recursion RecursionError
```

All four designs (while, while+break, for, recursive) give identical results for the four scenarios. Note that in the scenario *blank, wrong, blank, wrong, right* the correct OTP is accepted on attempt **3**, because the two blanks were not counted.

6. Answers to the questions

When is a `while` loop more natural than a `for` loop?

A `for` loop is natural when the number of passes is known in advance and nothing inside the loop changes it. Here the number of **counted** attempts is fixed at three, but the number of **passes** is not, because blank entries are skipped and, after the changed condition, extra attempts may be added. The loop is controlled by a condition (`attempts < allowed`) that the body changes, which is the definition of a `while` loop. With `for ... in range(3)` the blank entry needs an inner loop, which is clumsier.

What bug appears if the counter is updated in the wrong place?

If `attempts += 1` is written at the top of the loop (before the blank check), a blank entry is counted as an attempt. The program then locks the customer too early. Test: entries `['', '1', '2', '482913']` – the correct OTP is the **third real attempt**, yet:

Observed (from the output above)

```
correct while : ('ACCESS GRANTED', 3)
buggy while   : ('ACCOUNT LOCKED 30 min', 3)
```

The opposite mistakes are also possible: if the counter is never updated, or is updated only after a `continue` that skips it for every wrong entry, the loop never ends (infinite loop at the kiosk).

7. Changed condition – two extra attempts after a correct security answer

After three failed attempts the kiosk asks one security question. If it is answered correctly, **two more attempts** are allowed (once only). This is a new requirement that changes the limit **while the loop is running**.

- **Design A (while)** only needs the limit to be a variable: `allowed += 2`. The inner loop test `attempts < allowed` automatically uses the new limit. See `verify_extra` in the source code.
- **Design B (for with `range(3)`)** cannot change the limit of a running `range`; it would need an outer loop or a second `for _ in range(2)` loop plus copied code. The structure no longer fits the problem.
- **Decision:** the `while` design is kept and generalised; the `for` design is rejected because the number of passes is now decided during the run.

Case (entries: 1, 2, 3, 4, then the OTP ...)	Observed result
security answer correct; OTP typed as the 5th attempt	('ACCESS GRANTED', 5)
security answer wrong	('ACCOUNT LOCKED 30 min', 3)
security answer correct but five wrong tries (1 to 6 typed, OTP is 7th)	('ACCOUNT LOCKED 30 min', 5): bonus given only once

8. Alternative concept and complexity analysis – recursion

A recursive function `verify_rec(read, attempt, max_attempts)` calls itself for the next attempt instead of looping: the base case is `attempt > max_attempts` (lock), the other cases return success or recurse with `attempt + 1`. A blank entry calls the function again with the **same** attempt number. The complete code is in the source listing above.

a (attempts)	Loop time	Recursion time	Remark
3	1.3 µs	1.7 µs	real kiosk size: no practical difference
100	2.9 µs	19.4 µs	
900	27.6 µs	251.4 µs	recursion about 9 times slower (frame creation)
5000	155.5 µs	RecursionError	default limit is 1000 frames

- **Time:** O(a) for all versions – each attempt is processed once. Recursion has a larger constant (creating a stack frame per call).
- **Space:** the loop needs O(1) (two integers); recursion needs O(a) stack frames, and blank entries add frames too, because they recurse as well. Python stops recursion at depth 1000 (`sys.getrecursionlimit()`), shown by the `RecursionError` for a = 5000.
- **Conclusion:** for a security-critical kiosk a loop is preferable: its memory use does not depend on the input. With a = 3 recursion is correct and readable, but gives no advantage here. Timings were measured on CPython 3.12.3 and vary between computers.

9. Conclusion

The `while` loop with a counter updated after the blank check is the clearest correct design and adapts to the changed rule; the `for` design needs an inner loop and cannot extend its limit; recursion works but uses O(a) stack and is slower.

PYTHON PROGRAMMING · ASSIGNMENT 2 (CO2)

Q3 – Engineer a Vehicle Registration Number Validator

6 Marks · Bloom's Level: K4 – Analyse · Individual assignment · Submission date: 30/10/2026

Name: Dhaarani D	**Register Number:** _____
GitHub repository: _____	**Repository folder:** Task03/

1. Problem and rules

Operators type the same vehicle number in many ways (`tn 09 ab 1234`, `TN-09-AB-1234`, `Tn09Ab1234`). A valid number has **2 letters** (state), **2 digits** (district), **1 or 2 letters** (series) and **4 digits**. Only string operations are used: indexing, slicing, `len`, `upper`, `replace`, `strip`, `isalpha`, `isdigit`, `split`, `join`. No regular expressions are used in the solution.

2. Predictions before execution (six designed inputs)

Predictions were written down before the program was run. Two inputs are designed to be rejected.

#	Typed input	Predicted normalised form	Predicted decision	Reason
1	`tn 09 ab 1234`	TN09AB1234	valid → `TN 09 AB 1234`	spaces removed, upper case, 10 characters
2	`TN-09-AB-1234`	TN09AB1234	valid → `TN 09 AB 1234`	hyphens removed
3	`Tn09Ab1234`	TN09AB1234	valid → `TN 09 AB 1234`	only the case differs
4	`ka-01-m-0007`	KA01M0007	valid → `KA 01 M 0007`	one series letter, 9 characters
5	`TN 09 ABC 1234`	TN09ABC1234	**rejected**	three series letters, length 11
6	`TN 09 AB 12A4`	TN09AB12A4	**rejected**	last four are not all digits

3. Complete the Code – TODO 1 and TODO 2 completed

`normalise` removes the surrounding blanks with `strip()`, then spaces and hyphens with `replace`, then converts to upper case. `is_valid` first checks the length (9 or 10) and then checks each part by slicing. Because the series is everything between the first four and last four characters, `reg[4:-4]`, its length is 1 or 2 automatically once the total length is 9 or 10.

Task03/plate.py (complete program, including the formatting function)

```
"""Q3 - Vehicle registration number validator using only string operations (no regex)."""
import re, timeit

def normalise(text):
    """'tn 09 ab 1234' -> 'TN09AB1234' (spaces and hyphens removed, upper case)."""
    # TODO 1 done: remove spaces and hyphens, convert to upper case
    return text.strip().replace(" ", "").replace("-", "").upper()

def is_valid(reg):
    """Valid form: 2 letters, 2 digits, 1 or 2 letters, 4 digits (e.g. TN09AB1234)."""
    # TODO 2 done: length 9 or 10, then check every part with slicing + isalpha()/isdigit()
    if len(reg) not in (9, 10):
        return False
    return (reg[:2].isalpha()                # state
            and reg[2:4].isdigit()           # district
            and reg[4:-4].isalpha()          # series: 1 or 2 letters (length is forced by len check)
            and reg[-4:].isdigit())          # number

def format_plate(reg):
    """Format a valid normalised number as 'TN 09 AB 1234'."""
    return " ".join([reg[:2], reg[2:4], reg[4:-4], reg[-4:]])

for raw in ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234", "TN 22 1234 AB"]:
    reg = normalise(raw)
    print(f"{raw:15} -> {reg:11} {'valid' if is_valid(reg) else 'INVALID'}")

print("\n--- six designed inputs (predictions made before running) ---")
tests = [
    ("tn 09 ab 1234", "TN 09 AB 1234"),
```

```
    ("TN-09-AB-1234", "TN 09 AB 1234"),
    ("Tn09Ab1234",    "TN 09 AB 1234"),
    ("ka-01-m-0007",  "KA 01 M 0007"),
    ("TN 09 ABC 1234", None),          # 3 series letters -> length 11 -> reject
    ("TN 09 AB 12A4",  None),          # letter inside the number -> reject
]
for raw, expected in tests:
    reg = normalise(raw)
    got = format_plate(reg) if is_valid(reg) else None
    print(f"{raw!r:18} -> {str(got):15} expected {str(expected):15} {'OK' if got == expected else 'MISMATCH'}")

print("\n--- slicing on 9 and 10 character numbers ---")
for s in ("KA01M0007", "TN09AB1234"):
    print(f"{s} (len {len(s)}): s[-4:] = {s[-4:]!r}, s[4:-4] = {s[4:-4]!r}")

print("\n--- immutability ---")
text = "tn09"
up = text.upper()
print("text =", text, "| text.upper() returned", up, "| text still", text)
try:
    text[0] = "T"
except TypeError as e:
    print("text[0] = 'T' ->", type(e).__name__ + ":", e)
print("id before/after upper() differ:", id(text) != id(up))

# ---------- Alternative: regular expression ----------
PATTERN = re.compile(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}")
def is_valid_regex(reg):
    return PATTERN.fullmatch(reg) is not None

import itertools, random
random.seed(1)
pool = "AB09 -x"
samples = ["".join(random.choice("ABT09") for _ in range(random.choice((8, 9, 10, 11)))) for _ in range(20000)]
samples += ["TN09AB1234", "KA01M0007", "TN9AB1234", "TN221234AB"]
same = all(is_valid(s) == is_valid_regex(s) for s in samples)
print("\nregex and slicing versions agree on", len(samples), "inputs:", same)
n = 200_000
t1 = timeit.timeit(lambda: is_valid("TN09AB1234"), number=n) / n * 1e9
t2 = timeit.timeit(lambda: is_valid_regex("TN09AB1234"), number=n) / n * 1e9
print(f"slicing version: {t1:6.1f} ns per call   regex version: {t2:6.1f} ns per call")
```

4. Observed results

Program output

```
tn 09 ab 1234   -> TN09AB1234  valid
KA-01-M-0007    -> KA01M0007   valid
TN9AB1234       -> TN9AB1234   INVALID
TN 22 1234 AB   -> TN221234AB  INVALID

--- six designed inputs (predictions made before running) ---
'tn 09 ab 1234'    -> TN 09 AB 1234   expected TN 09 AB 1234   OK
'TN-09-AB-1234'    -> TN 09 AB 1234   expected TN 09 AB 1234   OK
'Tn09Ab1234'       -> TN 09 AB 1234   expected TN 09 AB 1234   OK
'ka-01-m-0007'     -> KA 01 M 0007    expected KA 01 M 0007    OK
'TN 09 ABC 1234'   -> None            expected None            OK
'TN 09 AB 12A4'    -> None            expected None            OK

--- slicing on 9 and 10 character numbers ---
KA01M0007 (len 9): s[-4:] = '0007', s[4:-4] = 'M'
TN09AB1234 (len 10): s[-4:] = '1234', s[4:-4] = 'AB'

--- immutability ---
text = tn09 | text.upper() returned TN09 | text still tn09
text[0] = 'T' -> TypeError: 'str' object does not support item assignment
id before/after upper() differ: True

regex and slicing versions agree on 20004 inputs: True
slicing version:  235.9 ns per call   regex version:  195.4 ns per call
```

The first four lines are **identical to the expected output** given in the assignment. All six designed inputs produced the predicted result (`OK`). The added function `format_plate` joins the four slices with single spaces, giving `TN 09 AB 1234` for a valid number.

5. Inquiry – immutability and slicing

Why are strings immutable, and why does `text.upper()` not change `text`?

A Python `str` cannot be modified after it is created; no operation changes its characters in place (`text[0] = 'T'` raises `TypeError: 'str' object does not support item assignment`, shown above).

Immutability lets strings be shared safely, be used as dictionary keys (their hash never changes) and be cached or reused by the interpreter. Therefore every string method such as `upper()`, `replace()` or `strip()` builds and **returns a new string** and leaves the original untouched. The output shows `text = tn09`, `text.upper()` returned `TN09` and `text` is still `tn09`; the two objects have different ids. To keep the result it must be assigned: `text = text.upper()`.

What do `s[-4:]` and `s[4:-4]` return?

Number	Length	`s[-4:]`	`s[4:-4]`	Meaning
KA01M0007	9	'0007'	'M'	last four characters = number; middle = 1 series letter
TN09AB1234	10	'1234'	'AB'	middle = 2 series letters

`s[-4:]` counts from the end and takes the last four characters; `s[4:-4]` starts at index 4 (after state and district) and stops 4 characters before the end, so it is exactly the series part, whatever its length. Slices never raise `IndexError`, they just give a shorter string, which is why the length is checked first.

Limitation noticed while testing: `isalpha()` and `isdigit()` also accept non-English letters and some special digit characters (for example superscript digits). For a production toll system an extra `isascii()` check, or the regular expression with `[A-Z]` and `[0-9]`, would be needed.

6. Alternative concept and complexity analysis – regular expression

The same check with a regular expression is a single line (the pattern is compiled once):

Regular-expression version

```
PATTERN = re.compile(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}")
def is_valid_regex(reg):
    return PATTERN.fullmatch(reg) is not None
```

Both versions were run on 20,004 inputs (20,000 random strings of 8–11 characters and the four given numbers) and gave **the same answer for every input**. Measured time for the valid number `TN09AB1234`: slicing version **235.9 ns** per call, regular-expression version **195.4 ns** per call (CPython 3.12.3; will differ on another computer).

Aspect	Slicing + string methods	Regular expression
Time	O(L): length check O(1), then each slice and `isalpha`/`isdigit` touches each character once, so L ≤ 10 characters	O(L): the pattern is matched once left to right; no backtracking is needed for this pattern
Space	O(L) for the small slices created (each slice copies its characters)	O(1) extra for matching (the compiled pattern is created once)
Readability	step-by-step, easy to debug, each rule is a separate visible check	very short, but the pattern must be understood by the reader
Ease of change	changing a rule (say 3 series letters) means editing indexes and the length test – risk of off-by-one mistakes	change `{1,2}` to `{1,3}` – one place

For fixed-length numbers L ≤ 10, both are effectively constant time. The regular expression is shorter and easier to change; the string-method version needs no extra module and is the right choice when the task specifically asks to practise indexing and slicing.

7. Conclusion

`normalise`, `is_valid` and `format_plate` give the expected output, accept all spellings of a valid number and reject the invalid ones. The key ideas are that every string method returns a new string (immutability) and that `s[4:-4]` extracts the series part whatever its length.

PYTHON PROGRAMMING · ASSIGNMENT 2 (CO2)

Q4 – Building Long Text: + or join()?

6 Marks · Bloom's Level: K4 – Analyse · Individual assignment · Submission date: 30/10/2026

Name: Dhaarani D	**Register Number:** _____
GitHub repository: _____	**Repository folder:** Task04/

1. Problem

A mobile operator builds each monthly statement by adding one line per call or SMS to a string. Heavy users have more than 100,000 lines and statement generation has become slow. The same statement is built three ways, as required: (a) repeated `+=` inside a loop, (b) a list of parts followed by `"".join()` and (c) an f-string for each line. `io.StringIO` is added as the alternative concept.

2. Prediction before measuring

- Strings are immutable, so `text += line` must create a new string and copy the old text each time. For n lines of about 15 characters, the total characters copied is about 15·(1 + 2 + … + n) = $O(n^2)$. Expected: (a) is the slowest and its time grows about 100 times when n grows 10 times.
- `join()` first collects the parts, computes the final length once and copies each part once: O(n). Expected: (b) and (c) are fastest, growing about 10 times when n grows 10 times.
- Expected at 200,000 lines: (a) is many times slower than (b) and (c). **This prediction turned out to be only partly right** – see the measurements and the explanation about the CPython optimisation below.

3. Implementations and measurement program

Task04/measure.py – (a) +=, (b) list + join, (c) f-string per line + join, StringIO, defeated +=

```
"""Q4 - measurements at 1,000 / 10,000 / 200,000 lines, 3 runs each, plus the case where += optimisation is
    defeated."""
import time, io, tracemalloc, json

def lines(n):
    return [f"Item{i}:Rs{i % 50}" for i in range(n)]

def a_plus(n):                      # (a) repeated +=
    text = ""
    for i in range(n):
        text += f"Item{i}:Rs{i % 50}" + ";"
    return text

def b_join(n):                      # (b) list of parts + join
    parts = []
    for i in range(n):
        parts.append(f"Item{i}:Rs{i % 50}")
    return ";".join(parts) + ";"

def c_fstring_join(n):              # (c) f-string for each line (line already contains ';'), then join
    return "".join([f"Item{i}:Rs{i % 50};" for i in range(n)])

def d_stringio(n):                  # alternative: io.StringIO
    buf = io.StringIO()
    for i in range(n):
        buf.write(f"Item{i}:Rs{i % 50};")
    return buf.getvalue()

def e_plus_defeated(n):             # += optimisation defeated: text + p + ';' creates a temporary copy
    text = ""
    for i in range(n):
        text = text + f"Item{i}:Rs{i % 50}" + ";"
    return text

def timed(fn, n, runs=3):
    ts = []
    for _ in range(runs):
        t = time.perf_counter(); out = fn(n); ts.append(time.perf_counter() - t)
    return sum(ts) / runs, out

sizes = [1_000, 10_000, 200_000]
methods = [("a: +=", a_plus), ("b: list+join", b_join), ("c: f-string+join", c_fstring_join), ("StringIO",
    d_stringio)]
results = {}
```

```
print(f"{'method':20}" + "".join(f"{n:>14,}" for n in sizes) + "   (average of 3 runs, seconds)")
for name, fn in methods:
    row = []
    for n in sizes:
        t, out = timed(fn, n)
        row.append(t)
    results[name] = row
    print(f"{name:20}" + "".join(f"{t:14.5f}" for t in row))

# all methods build identical text?
ref = a_plus(10_000)
print("\nall four give identical text (n=10,000):", all(fn(10_000) == ref for _, fn in methods))

# defeated optimisation: smaller sizes because it is quadratic
d_sizes = [1_000, 10_000, 50_000, 100_000]
print("\n+= with the optimisation defeated (text = text + p + ';'):")
defeated = []
for n in d_sizes:
    t, _ = timed(e_plus_defeated, n, runs=1)
    t2, _ = timed(a_plus, n, runs=1)
    defeated.append((n, t, t2))
    print(f"  n={n:>7,}: defeated {t:8.4f} s   plain += {t2:8.4f} s")

# memory (peak) at 200,000 lines
print("\npeak extra memory at 200,000 lines (tracemalloc):")
mem = {}
for name, fn in [("+=", a_plus), ("list+join", b_join), ("StringIO", d_stringio)]:
    tracemalloc.start(); out = fn(200_000); cur, peak = tracemalloc.get_traced_memory(); tracemalloc.stop()
    mem[name] = peak
    print(f"  {name:10} peak = {peak/1e6:6.2f} MB   (result {len(out)/1e6:.2f} MB)")

json.dump({"sizes": sizes, "results": results, "defeated": defeated, "mem": mem}, open("Task04/data.json", "w"))
```

4. Measured results

Each time is the **average of 3 runs** measured with `time.perf_counter()`. The time includes creating each line (the f-string), because that is part of building a statement.

Program output

```
method                      1,000        10,000       200,000   (average of 3 runs, seconds)
a: +=                     0.00017       0.00151       0.02931
b: list+join              0.00013       0.00128       0.03020
c: f-string+join          0.00013       0.00119       0.02884
StringIO                  0.00014       0.00127       0.02669

all four give identical text (n=10,000): True

+= with the optimisation defeated (text = text + p + ';'):
  n=  1,000: defeated   0.0003 s   plain +=   0.0002 s
  n= 10,000: defeated   0.0311 s   plain +=   0.0019 s
  n= 50,000: defeated   1.5755 s   plain +=   0.0106 s
  n=100,000: defeated   9.9223 s   plain +=   0.0276 s

peak extra memory at 200,000 lines (tracemalloc):
  +=         peak =   3.05 MB   (result 3.05 MB)
  list+join  peak =  18.77 MB   (result 3.05 MB)
  StringIO   peak =   6.86 MB   (result 3.05 MB)
```

Method	1,000 lines	10,000 lines	200,000 lines	Growth for 20× more lines (10,000 → 200,000)
(a) +=	0.00017 s	0.00151 s	0.02931 s	19.4×
(b) list + `join`	0.00013 s	0.00128 s	0.03020 s	23.6×
(c) f-string per line + `join`	0.00013 s	0.00119 s	0.02884 s	24.2×
`io.StringIO`	0.00014 s	0.00127 s	0.02669 s	21.0×

Observation: with CPython 3.12 the plain += loop did **not** show quadratic growth: 20 times more lines took about 19 times longer, the same as `join`. At this size, the time is dominated by creating the f-strings and running the loop, not by copying. All four methods built identical text (checked).

When the += optimisation is defeated

The statement `text = text + p + ";"` does the same job but evaluates `text + p` first as a temporary value. Now the string is referenced by a variable **and** the interpreter's stack, so it cannot be extended in place and the whole text is copied for every line.

Lines	`text = text + p + ';'`	`text += p + ';'`	Ratio
1,000	0.0003 s	0.0002 s	1.5×
10,000	0.0311 s	0.0019 s	16×
50,000	1.5755 s	0.0106 s	149×
100,000	9.9223 s	0.0276 s	360×

Between 10,000 and 100,000 lines (10× more) the defeated version became **319 times slower** (0.0311 s → 9.9223 s), which agrees with the predicted O(n²) behaviour (pure quadratic copying would give about 100×; the measured ratio is larger and I did not investigate the extra).

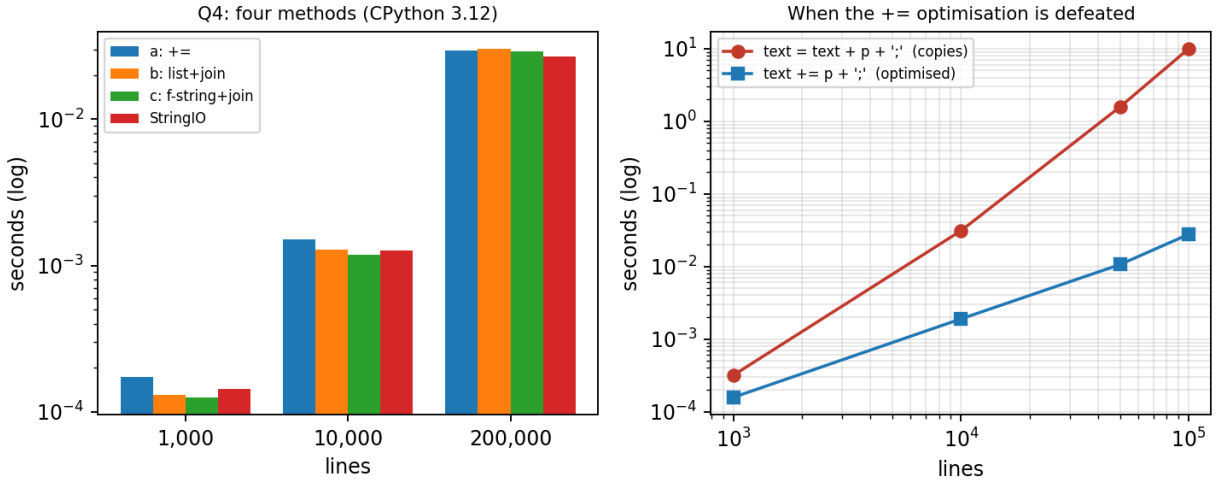


Figure 1 – Left: the four methods at three sizes. Right: the quadratic cost appears when the += optimisation does not apply.

5. Complete the Code – TODO 1

`build_join` builds the same text with one `str.join()` call. The trailing `;` after the last item is added once, so the text is identical to the `+=` version (length 3,048,890, equal to the expected output).

Task04/build.py

```
"""Q4 - Building long text: += vs join() vs f-string lines vs StringIO; times, memory, immutability."""
import time, io, tracemalloc, json

items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]

def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text

def build_join(parts):
    # TODO 1 done: single str.join() call (the trailing ';' keeps the text identical to build_plus)
    return ";".join(parts) + ";"

start = time.perf_counter(); a = build_plus(items); t_plus = time.perf_counter() - start
start = time.perf_counter(); b = build_join(items); t_join = time.perf_counter() - start
print("same text:", a == b, "| length:", len(b))
print(f"+= time: {t_plus:.4f} s   join time: {t_join:.4f} s")
```

Program output (run 1)

```
same text: True | length: 3048890
+= time: 0.0144 s   join time: 0.0323 s
```

The program was run three times:

Run	`+=` time	`join` time
1	0.0144 s	0.0323 s
2	0.0112 s	0.0054 s
3	0.0126 s	0.0052 s
Average	**0.0127 s**	**0.0143 s**

The first `join` run (0.0323 s) was much slower than the next two (about 0.005 s). The most likely reason is that the first run had to obtain fresh memory from the operating system for a 3 MB result, but I did not isolate this. **Honest reading of the result:** on this computer the average `join` time is not better than `+=` because of that one slow run; the two later runs show `join` about 2.3 times faster than `+=` (0.0053 s versus 0.0119 s). Times differ from one computer to another.

6. Inquiry – immutability and why `join()` is still recommended

Immutability. A `str` cannot be changed after creation. `text += p` therefore means *create a new string of length len(text)+len(p), copy both into it, and make* `text` *refer to it*. Done n times, the characters copied are 1 + 2 + … + n line-lengths, which is $O(n^2)$. `join()` receives all parts at once, adds up their lengths, allocates the result once and copies each part once: O(n).

The CPython optimisation. When the left side of `+=` is a local variable that holds the **only** reference to the string, CPython can grow that string in place (resizing its memory block, which is usually cheap) instead of copying. That is why `+=` was fast in the first experiment. But:

- it is an **implementation detail** of CPython, not a promise of the Python language; other implementations (for example PyPy or Jython) do not guarantee it;
- it only works when there is exactly one reference. It fails if the string is also stored in another variable, an attribute, a list or a dictionary, or when the code is written as `text = text + a + b` (shown above: 360 times slower at 100,000 lines);
- small changes by a later programmer can silently switch the optimisation off, and the program becomes quadratic.

`join()` is O(n) on every implementation and in every situation, and it states the intention clearly. For this reason `join()` (or `StringIO`) remains the recommended practice, even though `+=` can be fast in simple CPython loops.

7. Alternative concept and complexity analysis – file writing or `io.StringIO`

Instead of collecting parts, each line is **written immediately**: to an `io.StringIO` buffer (then `getvalue()` once) or directly to a file with `file.write(line)`. For a statement that will be saved or sent anyway, writing straight to the file avoids keeping the whole text in memory.

StringIO version (from measure.py)

```
def d_stringio(n):
    buf = io.StringIO()
    for i in range(n):
        buf.write(f"Item{i}:Rs{i % 50};")
    return buf.getvalue()
```

Approach	Time	Extra memory (measured peak at 200,000 lines, `tracemalloc`)
+= (optimised)	O(n) in CPython when optimised; up to $O(n^2)$ characters copied otherwise	3.05 MB (just the result)
list + `join`	O(n)	18.77 MB: the list of 200,000 small string objects exists besides the 3.05 MB result, so memory is about 6 times the result
`io.StringIO`	O(n), amortised: the buffer grows in blocks	6.86 MB
direct `file.write`	O(n)	O(1) beyond the file buffer (not measured here)

The result: `join` is the fastest and clearest when all parts are already available; `StringIO` or direct file output uses much less memory than collecting a list and is preferable for very large statements that are written out line by line.

8. Conclusion

In theory `+=` is O(n²) and `join()` is O(n). In practice CPython hides the cost of `+=` in a simple single-reference loop (19.4× time for 20× lines), but the cost becomes real as soon as the optimisation cannot apply (319× time for 10× lines). `join()` is dependable in all cases and is therefore the right choice for statements with more than 100,000 lines.

PYTHON PROGRAMMING · ASSIGNMENT 2 (CO2)

Q5 – Functions, Parameters and Scope

7 Marks · Bloom's Level: K4 – Analyse · Individual assignment · Submission date: 30/10/2026

Name: Dhaarani D	**Register Number:** _____
GitHub repository: _____	**Repository folder:** Task05/

1. Problem and the given principle

The canteen program has `add_item(item, order=[])`. Items from the previous customer appear in the next customer's order. The only principle given is: **a default value is created once, when the function is defined, not each time the function is called.**

2. Explaining the bug

- When Python executes the `def` line it evaluates `order=[]` **once** and stores that one list object inside the function object (in its `__defaults__` attribute).
- A call without `order` uses that same stored list. `order.append(item)` **changes the list in place** (a list is mutable) and so changes the stored default.
- The next customer's call without `order` gets the same, already filled list, so the orders accumulate.

Program output

```
['Tea', 'Vada'] ['Tea', 'Vada']
['Tea'] ['Vada']

add_item_buggy.__defaults__ after the calls: (['Tea', 'Vada'],)
add_item.__defaults__                      : (None,)

--- three customers ---
customer 1: ['Tea', 'Vada']  subtotal=22  gst=1.1  total=23.1
customer 2: ['Coffee']  subtotal=15  gst=0.75  total=15.75
customer 3: ['Samosa', 'Tea', 'Tea']  subtotal=40  gst=2.0  total=42.0
orders independent: True

--- parameter kinds ---
positional        : 31.5
keyword           : 31.5
default qty       : 10.5
keyword-only gst  : 30.0

--- scope ---
without global -> UnboundLocalError: cannot access local variable 'orders_served' where it is not associated with
    a value
orders_served (global) = 2
closure counter nonlocal = 3
local x is not visible outside -> NameError: name 'x' is not defined

--- alternatives ---
explicit : ['Tea'] ['Vada']
closure  : ['Tea'] ['Vada'] ['Tea', 'Samosa']
```

First line: `['Tea', 'Vada'] ['Tea', 'Vada']` – the two calls returned **the same list object**, which printed after both appends, so both show both items (it is not two copies). The expected output of the assignment is matched exactly.

`add_item_buggy.defaults` after the calls

The assignment writes `add_item_buggy.defaults`; in Python the real attribute name is `__defaults__` (with double underscores), which was printed:

From the program output

```
add_item_buggy.__defaults__ after the calls: (['Tea', 'Vada'],)
add_item.__defaults__                      : (None,)
```

The tuple of defaults of `add_item_buggy` now contains a list with both items: the default value itself was modified by the calls. For the corrected function the default is `None`, which is immutable and never changes.

3. Corrected design (TODO 1)

Completed function

```
def add_item(item, order=None):
    if order is None:        # TODO 1: new empty list for every call without an order
        order = []
    order.append(item)
    return order
```

`None` is used as a marker meaning *no order given*. A **new** list is created inside the function body, which runs at every call, so each customer starts with a separate, empty list. If the caller supplies an order list, that list is extended and returned.

4. Parameter kinds – when to use which

Kind	Example	Use when
Positional	`price_line("Tea", 3)`	few parameters whose order is obvious (item, quantity)
Keyword	`price_line(qty=3, item="Tea")`	many parameters, or the meaning of a value is not clear from its position; order can be changed freely
Default	`qty=1`	a sensible common value exists; the caller may leave it out. **Defaults must be immutable** (numbers, strings, None, tuples)
Keyword-only	`def f(item, qty=1, *, gst_rate=0.05)`	an option that should always be named at the call, so it cannot be mixed up

Observed: `price_line("Tea", 3)` = 31.5, `price_line(qty=3, item="Tea")` = 31.5, `price_line("Tea")` = 10.5, `price_line("Tea", 3, gst_rate=0.0)` = 30.0.

5. Returning more than one value

A function can return several values separated by commas; Python builds a **tuple** and the caller unpacks it:

Returning the subtotal and the GST

```
def bill(order, gst_rate=GST_RATE):
    subtotal = sum(menu[i] for i in order)
    return subtotal, round(subtotal * gst_rate, 2)   # tuple (total, GST)

sub, gst = bill(order)
```

6. Scope: local, global and nonlocal

Name type	Where it lives	Demonstration (from the program)	Observed
local	inside one function call only	`x = "local"` inside `local_demo()`; `print(x)` outside	NameError: name 'x' is not defined
global	module level, visible everywhere for reading	`orders_served = 0`; `serve_global()` declares `global orders_served` then `+= 1`	value 2 after two calls
nonlocal	in an enclosing function, shared with the inner function	`make_counter()` keeps `count`; inner `tick()` uses `nonlocal count`	third `tick()` returns 3

Why does changing a global inside a function need `global`, and why is it usually avoided?

Python decides at compile time that a name **assigned** anywhere inside a function is local to that function. In `serve_wrong()` the statement `orders_served += 1` therefore refers to a new local name that has no value yet, which gives `UnboundLocalError` (shown in the output). The keyword `global orders_served` tells Python to use the module-level name instead. Reading a global needs no keyword.

Writing to globals is usually avoided because: (i) any function can change the value, so the cause of a wrong value is hard to find; (ii) functions depend on hidden data and cannot be tested alone; (iii) it is not safe if two parts of a program use the same name. It is better to pass values in as parameters and **return** results, or to keep state in an object or a closure (`nonlocal`) that belongs to one purpose.

7. Test – three customers in sequence

Each customer starts with `add_item("...")` without an order and then adds more items by passing the returned list. Observed:

Customer	Order	Subtotal	GST (5%)	Total
1	['Tea', 'Vada']	22	1.1	23.1
2	['Coffee']	15	0.75	15.75
3	['Samosa', 'Tea', 'Tea']	40	2.0	42.0

The check `c1 is not c2 and c2 is not c3 and c1 == ['Tea','Vada'] and c2 == ['Coffee']` printed **True**: the three orders are different objects and none contains items of an earlier customer.

8. Complete source code

Task05/canteen.py

```
"""Q5 - Functions, parameters and scope: the mutable default argument bug."""

# ---------------- Given code with TODO 1 completed ----------------
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

def add_item(item, order=None):
    # TODO 1 done: new empty list when order is None, append, return
    if order is None:
        order = []
    order.append(item)
    return order

print(add_item_buggy("Tea"), add_item_buggy("Vada"))
print(add_item("Tea"), add_item("Vada"))

print("\nadd_item_buggy.__defaults__ after the calls:", add_item_buggy.__defaults__)
print("add_item.__defaults__                      :", add_item.__defaults__)

# ---------------- Test: three customers in sequence ----------------
print("\n--- three customers ---")
menu = {"Tea": 10, "Vada": 12, "Coffee": 15, "Samosa": 20}
GST_RATE = 0.05

def bill(order, gst_rate=GST_RATE):
    """Return TWO values (a tuple): subtotal and GST."""
    subtotal = sum(menu[i] for i in order)
    return subtotal, round(subtotal * gst_rate, 2)

c1 = add_item("Tea");    c1 = add_item("Vada", c1)
c2 = add_item("Coffee")
c3 = add_item("Samosa"); c3 = add_item("Tea", c3); c3 = add_item("Tea", c3)
for n, o in enumerate([c1, c2, c3], 1):
    sub, gst = bill(o)
    print(f"customer {n}: {o}  subtotal={sub}  gst={gst}  total={sub + gst}")
print("orders independent:", c1 is not c2 and c2 is not c3 and c1 == ["Tea", "Vada"] and c2 == ["Coffee"])

# ---------------- Positional, keyword and default parameters ----------------
print("\n--- parameter kinds ---")
def price_line(item, qty=1, *, gst_rate=0.05):
    return round(menu[item] * qty * (1 + gst_rate), 2)
print("positional        :", price_line("Tea", 3))
print("keyword           :", price_line(qty=3, item="Tea"))
print("default qty       :", price_line("Tea"))
print("keyword-only gst  :", price_line("Tea", 3, gst_rate=0.0))

# ---------------- Scope: local, global, nonlocal ----------------
print("\n--- scope ---")
orders_served = 0                     # global name

def serve_wrong():
    try:
        orders_served += 1            # assignment makes it LOCAL -> error
    except UnboundLocalError as e:
        print("without global ->", type(e).__name__ + ":", e)

def serve_global():
    global orders_served
    orders_served += 1                # changes the module-level name

def make_counter():
    count = 0                         # enclosing (nonlocal) name
    def tick():
        nonlocal count
        count += 1
```

```
        return count
    return tick

def local_demo():
    x = "local"                      # local name: invisible outside
    return x

serve_wrong(); serve_global(); serve_global()
print("orders_served (global) =", orders_served)
tick = make_counter(); tick(); tick()
print("closure counter nonlocal =", tick())
try:
    print(x)
except NameError as e:
    print("local x is not visible outside ->", type(e).__name__ + ":", e)

# ---------------- Alternative designs ----------------
# (1) caller always passes the list explicitly
def add_explicit(item, order):
    order.append(item)
    return order

# (2) closure that keeps its own list for each customer
def new_order():
    items = []
    def add(item):
        items.append(item)
        return list(items)
    return add

print("\n--- alternatives ---")
o1, o2 = [], []
print("explicit :", add_explicit("Tea", o1), add_explicit("Vada", o2))
a1, a2 = new_order(), new_order()
print("closure  :", a1("Tea"), a2("Vada"), a1("Samosa"))
```

9. Alternative concept and complexity analysis

Two other designs avoid the shared default list:

- **Caller passes the list explicitly:** `add_explicit(item, order)` has no default; each customer's list is created by the caller (`o1, o2 = [], []`). Observed: `['Tea'] ['Vada']`.
- **Closure:** `new_order()` creates an `items` list and returns an inner function `add(item)` that remembers it. Each call of `new_order()` creates a separate list, so customers cannot interfere. Observed: `a1('Tea')` → `['Tea']`, `a2('Vada')` → `['Vada']`, `a1('Samosa')` → `['Tea', 'Samosa']`.

Design	Time per item	Space per customer with m items	Remarks
default `None` + new list	O(1) amortised (list append)	O(m)	simple, familiar, the usual Python idiom
list passed explicitly	O(1)	O(m), owned by the caller	most transparent; but every caller must remember to create a list
closure	O(1) for the append; my add returns `list(items)`, a copy, which costs O(m) per call (return `items` itself to stay O(1))	O(m) plus a small constant for the inner function and its cell	state is protected from outside access; good when an order has several operations

Time is O(1) per item for all designs, since only an append is done; the space per customer is proportional to the number of items. The designs differ in **safety and clarity**, not in speed.

10. Conclusion

The bug is caused by a mutable default value created once at definition time. Using `None` as the default and creating the list inside the function removes it; the three-customer test shows independent orders. Parameters, tuple returns and scope rules were demonstrated with executed examples.

PYTHON PROGRAMMING · ASSIGNMENT 2 (CO2)

Q6 – Recursion and Its Limits

8 Marks · Bloom's Level: K5 – Evaluate · Individual assignment · Submission date: 30/10/2026

Name: Dhaarani D	**Register Number:** _____
GitHub repository: _____	**Repository folder:** Task06/

1. Problem

A temple has n steps and a devotee climbs 1 or 2 steps at a time. The trust wants the number of different ways to reach the top for staircases of up to 1,000 steps.

2. Recursive design

Base case and recursive case

- **Last move:** the final move onto step n was either a 1-step move from step n−1 or a 2-step move from step n−2. These two groups of ways do not overlap, so they can be added.
- **Recursive case:** `ways(n) = ways(n-1) + ways(n-2)` for n ≥ 2.
- **Base cases:** `ways(0) = 1` (one way to stay at the bottom: do nothing) and `ways(1) = 1` (a single 1-step move). The program writes both as `if n <= 1: return 1`.
- Each call works on a smaller n, so the recursion always reaches a base case.

Why plain recursion repeats work – call tree for n = 5

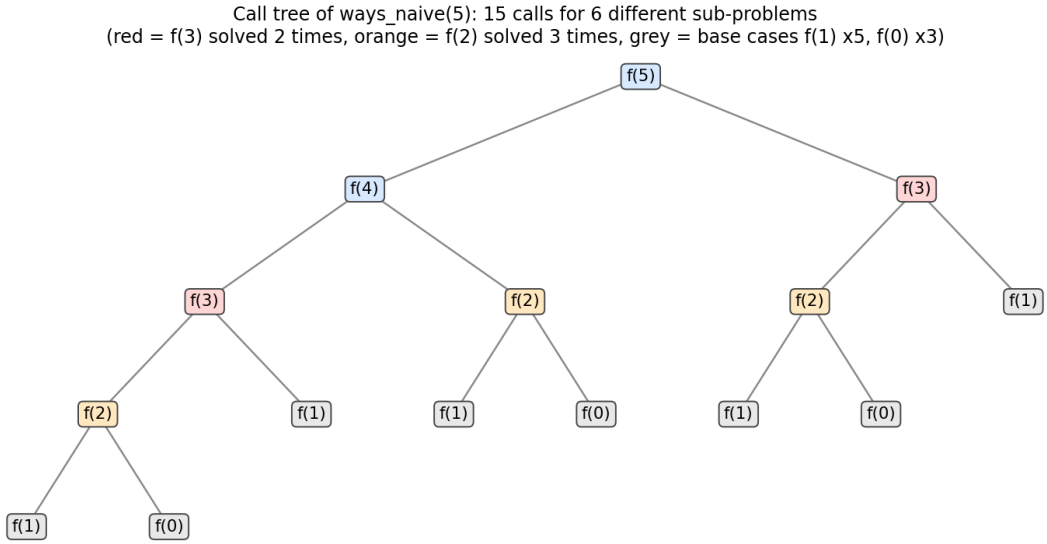


Figure 1 – Call tree of ways_naive(5). f(3) is solved twice, f(2) three times, f(1) five times and f(0) three times: 15 calls for only 6 different sub-problems.

Because `ways(n-1)` and `ways(n-2)` both need `ways(n-3)` and so on, the same sub-problems appear again and again in different branches. Total calls for ways_naive(n) = 2·ways(n) − 1 (for n = 5: 2·8 − 1 = 15, matching the tree), so the number of calls grows like ways(n) itself, which is about 1.618^n.

Memoisation removes the repetition

Memoisation stores the result of each sub-problem the first time it is computed (in a dictionary, or automatically by `functools.lru_cache`). A later request for the same n is answered from the cache without recursion. Every n from 0 to N is then computed exactly once, so the number of real calls is N + 1.

3. Complete the Code – TODO 1 and source code

Task06/steps.py

```
"""Q6 - Counting ways to climb temple steps: naive recursion, memoisation, loop; recursion limit."""
import sys, time, json
from functools import lru_cache
```

```
calls = 0

def ways_naive(n):
    """Ways to climb n temple steps taking 1 or 2 steps at a time."""
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)

@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1
    # TODO 1 done: same base case and recursive case as ways_naive, calling ways_memo
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)

def ways_loop(n):
    prev, cur = 1, 1                # ways(0), ways(1)
    for _ in range(n - 1):
        prev, cur = cur, prev + cur
    return cur

calls = 0; print("naive:", ways_naive(25), "calls =", calls)
calls = 0; print("memo :", ways_memo(25), "calls =", calls)

print("\n--- correctness by hand (n = 1..6) ---")
hand = {1: 1, 2: 2, 3: 3, 4: 5, 5: 8, 6: 13}
for n, h in hand.items():
    print(f"n={n}: hand={h} naive={ways_naive(n)} memo={ways_memo(n)} loop={ways_loop(n)}  {'OK' if h ==
        ways_naive(n) == ways_memo(n) == ways_loop(n) else 'MISMATCH'}")

print("\n--- call counts (prediction: naive calls = 2*ways(n) - 1, memo calls = n + 1) ---")
rows = []
for n in (10, 20, 25, 30):
    calls = 0; ways_naive(n); cn = calls
    ways_memo.cache_clear(); calls = 0; ways_memo(n); cm = calls
    pred = 2 * ways_loop(n) - 1
    rows.append((n, pred, cn, cm))
    print(f"n={n:2}: predicted naive={pred:>9,}  observed naive={cn:>9,}  memo={cm:3}  (n+1={n+1})")

print("\n--- naive time growth ---")
times = []
for n in (25, 30, 32, 35):
    t = time.perf_counter(); ways_naive(n); dt = time.perf_counter() - t
    times.append((n, dt)); print(f"n={n}: {dt:.3f} s")
print("ratio t(35)/t(30) =", round(times[-1][1] / times[1][1], 1), " (phi^5 =", round(1.618034 ** 5, 1), ")")

print("\n--- n = 1000 ---")
print("recursion limit:", sys.getrecursionlimit())
ways_memo.cache_clear()
try:
    ways_memo(1000)
    print("memo(1000) worked")
except RecursionError as e:
    print("ways_memo(1000) ->", type(e).__name__ + ":", e)

ways_memo.cache_clear()
old = sys.getrecursionlimit()
sys.setrecursionlimit(5000)
calls = 0; v = ways_memo(1000); c = calls
print(f"with setrecursionlimit(5000): ways_memo(1000) works, calls = {c}, digits = {len(str(v))}")
sys.setrecursionlimit(old)

t = time.perf_counter(); v2 = ways_loop(1000); dt = time.perf_counter() - t
print(f"loop version: ways(1000) has {len(str(v2))} digits, starts {str(v2)[:12]}..., time {dt*1e6:.1f} us, same
    as memo: {v == v2}")
print("naive for n=1000 is impossible: about phi^1000 =", f"{1.618034**1000:.2e}", "calls")

json.dump({"rows": rows, "times": times}, open("Task06/data.json", "w"))
```

4. Observed results

Program output

```
naive: 121393 calls = 242785
memo : 121393 calls = 26

--- correctness by hand (n = 1..6) ---
n=1: hand=1 naive=1 memo=1 loop=1  OK
n=2: hand=2 naive=2 memo=2 loop=2  OK
n=3: hand=3 naive=3 memo=3 loop=3  OK
n=4: hand=5 naive=5 memo=5 loop=5  OK
```

```
n=5: hand=8 naive=8 memo=8 loop=8  OK
n=6: hand=13 naive=13 memo=13 loop=13  OK

--- call counts (prediction: naive calls = 2*ways(n) - 1, memo calls = n + 1) ---
n=10: predicted naive=      177  observed naive=      177  memo= 11  (n+1=11)
n=20: predicted naive=   21,891  observed naive=   21,891  memo= 21  (n+1=21)
n=25: predicted naive=  242,785  observed naive=  242,785  memo= 26  (n+1=26)
n=30: predicted naive=2,692,537  observed naive=2,692,537  memo= 31  (n+1=31)

--- naive time growth ---
n=25: 0.013 s
n=30: 0.201 s
n=32: 0.438 s
n=35: 1.806 s
ratio t(35)/t(30) = 9.0  (phi^5 = 11.1 )

--- n = 1000 ---
recursion limit: 1000
ways_memo(1000) -> RecursionError: maximum recursion depth exceeded
with setrecursionlimit(5000): ways_memo(1000) works, calls = 1001, digits = 209
loop version: ways(1000) has 209 digits, starts 703303677114..., time 57.4 us, same as memo: True
naive for n=1000 is impossible: about phi^1000 = 9.72e+208 calls
```

The first two lines are **identical to the expected output** of the assignment (`naive: 121393 calls = 242785`, `memo : 121393 calls = 26`). The memoised function counted only 26 calls because `calls += 1` is executed only when the function body really runs, which is not when `lru_cache` returns a stored value.

5. Correctness check – n = 1 to 6 by hand

n	By hand	Working	naive	memo	loop
1	1	base case	1	1	1
2	2	1+1	2	2	2
3	3	2+1	3	3	3
4	5	3+2	5	5	5
5	8	5+3	8	8	8
6	13	8+5	13	13	13

For example, n = 3: 1+1+1, 1+2, 2+1 = 3 ways. All three programs agree with the hand values (the program printed `OK` for every n).

6. Growth investigation – counting function calls

Prediction: naive calls = 2·ways(n) − 1 and memoised calls = n + 1. With ways(10) = 89, ways(20) = 10,946, ways(25) = 121,393 and ways(30) = 1,346,269 (from the loop version) the predicted naive calls are 177, 21,891, 242,785 and 2,692,537.

n	Predicted naive calls	Observed naive calls	Observed memo calls (n + 1)
10	177	177	11
20	21,891	21,891	21
25	242,785	242,785	26
30	2,692,537	2,692,537	31

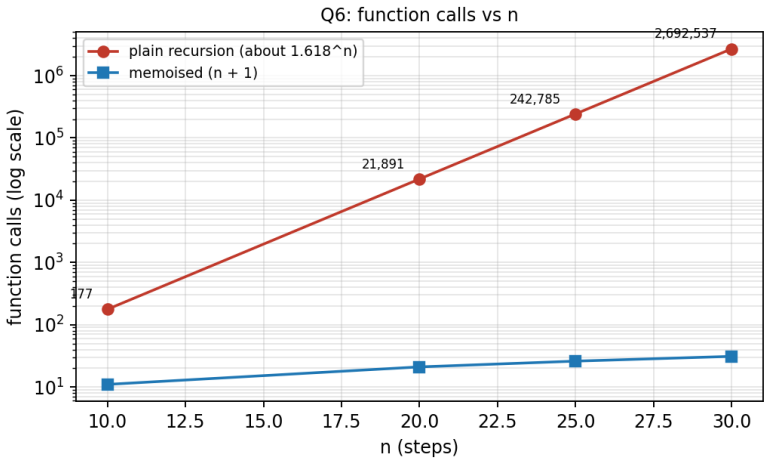


Figure 2 – Function calls against n on a log scale: the naive version grows exponentially, the memoised version linearly.

The observed counts match the prediction exactly. Each increase of n by 5 multiplies the naive calls by about 11 ($1.618^5 \approx 11.1$): 21,891 → 242,785 is 11.1×, and 242,785 → 2,692,537 is 11.1×. The measured **time** of the naive version showed the same trend: n = 30: 0.201 s, n = 32: 0.438 s, n = 35: 1.806 s; the ratio t(35)/t(30) was 9.0, near the expected 11.1 (timer noise and the cost of the very first measurements explain the difference). Extrapolating, n = 50 would take hours and n = 1,000 is impossible.

7. Deployment decision – n = 1,000 and `RecursionError`

Calling `ways_memo(1000)` with Python's default limit failed:

Observed

```
recursion limit: 1000
ways_memo(1000) -> RecursionError: maximum recursion depth exceeded
```

Explanation. `sys.getrecursionlimit()` returns 1000 – the maximum depth of nested Python calls, a safety limit that stops runaway recursion from using up the memory of the call stack. The first call `ways_memo(1000)` calls `ways_memo(999)`, which calls `ways_memo(998)`, and so on down to `ways_memo(1)`. **Nothing is in the cache yet** while going down; results are stored only on the way back. The chain is therefore about 1,000 calls deep (plus the frames of the main program and the `lru_cache` wrapper), which crosses the limit. Memoisation removes repeated *work* but not the *depth* of the recursion.

Options tested:

- **Raise the limit:** `sys.setrecursionlimit(5000)` – ways_memo(1000) then ran successfully with 1,001 calls and a 209-digit answer. This works, but each level of recursion uses memory, and a very high limit can crash the interpreter itself because the real stack is finite.
- **Rewrite with a loop:** `ways_loop(1000)` keeps only the last two values; it needed 57.4 µs, gave the same 209-digit number (starting 703303677114…) and has no recursion limit at all.

Decision: rewrite with a loop for the trust's program, and do not rely on raising the limit. The trust's requirement is simple (a number for each n up to 1,000 and possibly larger later). A loop is O(n) in time and O(1) in stack space for any n and cannot raise `RecursionError`. Raising the limit would be a fragile fix (it moves the failure from n ≈ 1,000 to n ≈ 5,000 and depends on a machine setting). The memoised recursive version is kept as a readable, correct reference to test the loop for n ≤ 900, where it works with the default limit. Using both together, a raised limit is acceptable only as a short temporary measure for the memoised version.

8. Alternative concept and complexity analysis – loop with two values

Loop that keeps only the last two values

```
def ways_loop(n):
    prev, cur = 1, 1                # ways(0), ways(1)
    for _ in range(n - 1):
        prev, cur = cur, prev + cur
```

```
    return cur
```

Version	Time	Extra space	n = 1,000
plain recursion	$O(\varphi^n)$, $\varphi \approx 1.618$ (exponential)	O(n) stack	**cannot be run**: about 9.7×10^{208} calls; at about 60 ns per call (measured speed for n = 35) it would take about 10^{194} years
memoised recursion	O(n)	O(n) cache + O(n) stack	RecursionError at the default limit; 1,001 calls and correct answer after raising the limit to 5,000
loop (two values)	O(n)	O(1) (two variables)	57.4 µs, correct 209-digit answer

All three give the same results for n ≤ 35 where all three can be run (see the correctness check). The numbers become very large: ways(1000) has 209 digits. Since Python integers can have any size, no overflow occurs, but adding such numbers takes time proportional to their length, a point used in Q7.

9. Conclusion

Plain recursion repeats work exponentially (242,785 calls for n = 25, 2,692,537 for n = 30). Memoisation reduces this to n + 1 calls but not the recursion depth, so n = 1,000 fails at the default limit. A loop with two variables gives the answer in O(n) time and O(1) space and is the right deployment choice.

PYTHON PROGRAMMING · ASSIGNMENT 2 (CO2)

Q7 – Diagnose, Redesign and Validate (Day-4 Design Challenge)

13 Marks · Bloom's Level: K5 – Evaluate · Individual assignment · Submission date: 30/10/2026

Name: Dhaarani D	**Register Number:** _____
GitHub repository: _____	**Repository folder:** Task07/

Program selected: the temple-steps counter of **Q6** (`ways_naive`). Reviewing the evidence from Q1–Q6, this program shows the largest limitation as the input grows: 242,785 calls for n = 25, 2,692,537 for n = 30, and n = 1,000 impossible. The redesign below is **not** a replacement by a built-in function: it is derived from the structure of the problem. All numbers come from `Task07/redesign.py` (complete listing at the end).

A. Diagnose the Bottleneck (2 marks)

- **Where most of the work is done:** nearly all of it is in the repeated recursive calls to the same sub-problems. For n = 25 the program makes 242,785 calls, but there are only **26 different sub-problems** (n = 0 … 25). `ways(1)` alone is solved 75,025 times.
- **Which count grows with the input:** the number of function calls, calls(n) = 2·ways(n) − 1, which grows like 1.618^n: 177 (n = 10), 21,891 (n = 20), 242,785 (n = 25), 2,692,537 (n = 30), 29,860,703 (n = 35). Measured time follows: 1.7 s at n = 35. A second growing quantity is the recursion depth, n, which stops the program near n = 1,000 (`RecursionError`).
- **Is work repeated?** Yes. Counting how often each sub-problem is solved for n = 5 gave f(5):1, f(4):1, f(3):2, f(2):3, f(1):5, f(0):3 (15 calls, 6 different problems; see the call tree in Q6). For n = 25, **99.989 %** of the calls repeat a sub-problem that was already solved (1 − 26/242,785).

Evidence (from the program output)

```
sub-problem repeat counts for n = 5: {5: 1, 4: 1, 3: 2, 2: 3, 1: 5, 0: 3}
n = 25: total calls = 242785 | distinct sub-problems = 26 | ways(1) alone solved 75025 times | repeated work =
    99.9893%
```

B. Define the Design Objective (2 marks)

Derived from the diagnosis (exponential calls, repeated sub-problems, depth n), the redesign must meet **all** of these measurable targets:

- **O1 – Work:** for n = 30, at most n − 1 = 29 basic operations (additions) instead of 2,692,537 function calls, that is, a number of operations that is linear in n.
- **O2 – Depth:** no recursion at all, so n = 1,000 runs without `RecursionError` and without changing `sys.setrecursionlimit`.
- **O3 – Output:** all 1,000 values ways(1) … ways(1000) that the trust needs are produced in under 10 ms.
- **O4 – Memory:** O(1) extra memory for a single n (no cache), with the same results as the original for every n that the original can compute.

C. Design the Revised Solution (4 marks)

What changed. Instead of starting from n and going *down* through a tree of calls, the new design starts from the base values and goes *up* from ways(1) to ways(n) in one loop (bottom-up dynamic programming).

Property exploited. (1) *Overlapping sub-problems:* the same ways(i) is needed many times, so each should be computed once. (2) *Dependence on only the two previous values:* ways(i) needs just ways(i−1) and ways(i−2), so after computing ways(i) the older value ways(i−2) is never needed again and **need not be stored**. Two variables are enough.

Pseudocode

```
function ways_dp(n):
    prev <- 1                    # ways(0)
```

```
    cur  <- 1                      # ways(1)
    repeat n - 1 times:
        (prev, cur) <- (cur, prev + cur)     # one addition per step
    return cur

function ways_table(N):            # all answers 1..N for the trust
    out <- [1]
    prev <- 1 ; cur <- 1
    repeat N - 1 times:
        (prev, cur) <- (cur, prev + cur)
        append cur to out
    return out
```

Flowchart (text form)

```
   start
     |
   prev = 1, cur = 1
     |
   i = 1 ----------------------+
     |                         |
   i < n ?  --no--> return cur |
     | yes                     |
   prev, cur = cur, prev + cur |
     |                         |
   i = i + 1 ------------------+
```

Extra memory. For one value of n the redesign needs only two variables: O(1). If the whole table 1…N is wanted, the list of N values is the output, so O(N) is needed (84.7 KB for N = 1,000). **New trade-offs:** (i) the code no longer mirrors the mathematical definition, so correctness must be checked against the simple original (done for all n shown below); (ii) the intermediate values cannot be reused later for a different n (the memoised version could reuse its cache; the table version can); (iii) the numbers have up to 209 digits, so each addition is not a constant-time operation (explained under F).

D. Predict Before Implementing (1 mark)

- **Operation count:** original calls = 2·ways(n) − 1; redesign additions = n − 1. For n = 30: 2,692,537 calls versus 29 additions, an improvement factor of 2,692,537 / 29 ≈ **92,800**.
- **Time:** the improvement should be of the same order, since one loop pass costs about as much as one call or less; I predict **at least 10,000×** at n = 30 and a larger factor at n = 35, with the factor smaller at n = 10 because the fixed cost of calling `ways_dp` itself dominates there.
- **Depth and memory:** recursion depth 0 instead of n; extra memory 2 variables. n = 1,000 will run, taking about 1000 additions.
- **Growth of the redesign:** time roughly linear in n (doubling n doubles time) – to be checked, because the numbers become big.

E. Implement and Validate the Redesign (2 marks)

Both versions were run on the same inputs. 'Original time' is a single run; 'redesign time' is the average of 2,000 runs because one run is too short to measure. Results were compared with == (column *equal*).

Program output

```
sub-problem repeat counts for n = 5: {5: 1, 4: 1, 3: 2, 2: 3, 1: 5, 0: 3}
n = 25: total calls = 242785 | distinct sub-problems = 26 | ways(1) alone solved 75025 times | repeated work =
    99.9893%

 n | original calls | redesign additions | original time | redesign time | speed-up | equal
10 |            177 |                  9 |     0.000012s |       0.34 us |       34x | True
20 |         21,891 |                 19 |     0.001199s |       0.59 us |    2,039x | True
25 |        242,785 |                 24 |     0.013017s |       0.70 us |   18,638x | True
30 |      2,692,537 |                 29 |     0.144247s |       0.83 us |  173,526x | True
35 |     29,860,703 |                 34 |     1.708586s |       0.95 us | 1,793,093x | True

Recursion depth: original needs depth n; redesign needs none.
recursion limit: 1000
original ways_naive(1500) -> RecursionError (it would also need ~1e313 calls)
redesign ways_dp(1000): 999 additions, 209-digit answer, no recursion limit involved

full table ways(1..1000): 1000 values in 0.182 ms, last value has 209 digits
naive calls for table up to n=30 only: 7,049,122 (table to 1000 is impossible)
peak memory: rolling variables 0.5 KB, full table 84.7 KB

redesign time growth (average of 200 runs):
  n=  100:      2.9 us
```

```
n=  200:      6.0 us
n=  400:     14.0 us
n=  800:     32.5 us
n= 1600:     74.8 us
```

n	Original calls	Redesign additions	Original time	Redesign time	Speed-up	Same answer
10	177	9	0.000012 s	0.34 µs	34×	yes
20	21,891	19	0.001199 s	0.59 µs	2,039×	yes
25	242,785	24	0.013017 s	0.70 µs	18,638×	yes
30	2,692,537	29	0.144247 s	0.83 µs	173,526×	yes
35	29,860,703	34	1.708586 s	0.95 µs	1,793,093×	yes
1,000	impossible (≈ 10^{209})	999	–	57 µs (Q6 run)	–	209-digit value, same as memoised version

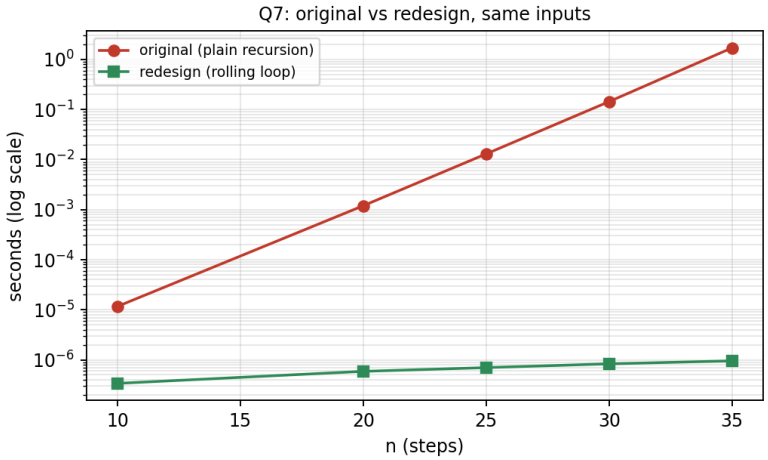


Figure 1 – Time against n (log scale): the original grows exponentially, the redesign stays almost flat.

Time for the original at n = 30 was 0.201 s in the Q6 run and 0.144 s here; single timings vary between runs, which does not change any conclusion because the ratio is above 100,000.

F. Defend the Final Design (2 marks)

Was the objective achieved?

Objective	Target	Measured	Met?
O1 Work	≤ 29 operations at n = 30	29 additions (2,692,537 calls before)	yes
O2 Depth	n = 1,000 without RecursionError	999 additions, no recursion, no limit change	yes
O3 Output	table 1...1,000 under 10 ms	0.182 ms	yes
O4 Memory	O(1) for a single n; same results	0.5 KB peak (84.7 KB for the full table); all answers equal	yes

Size of the improvement

At n = 30, 92,800 times fewer operations and 173,526 times less time; at n = 35 about 1.8 million times faster. The redesign changes the growth from exponential (1.618^n) to linear in the number of additions.

New costs and what the prediction missed

- **Big-number cost:** the redesign's time was not exactly linear. n = 100 took 2.9 µs, n = 200 6.0 µs, n = 400 14.0 µs, n = 800 32.5 µs, n = 1,600 74.8 µs: doubling n **more than doubles** the time (about 2.3× per doubling). The reason is that the numbers have about 0.21·n digits, so every addition handles longer numbers; the total work is closer to $O(n^2)$ digit operations. My prediction of 'roughly linear' was therefore only valid for the number of additions, not for the time at large n. This is still tiny compared with the original.
- **Readability and reuse:** the loop hides the mathematical definition, and the intermediate values are not available to answer other questions unless the table version is used (O(N) memory).

- **Correctness risk:** a new implementation can contain an off-by-one error; this was controlled by comparing with the original for n = 10, 20, 25, 30, 35 (and with the hand values 1...6 in Q6).

Where the original may still be preferable

For very small n, or in teaching, the original is the shortest, most direct translation of the definition and is useful as a **test oracle** to check faster versions (it was used that way here). The memoised recursive version is preferable when the sub-problems depend on many earlier values in a less regular way (so that two variables are not enough) or when the same function must be called later with arbitrary n and the cache reused. For the temple trust's task, where the answer for n depends only on the previous two and all n up to 1,000 are needed, the bottom-up loop is the better design.

Conclusion

The diagnosis (exponential, repeated, deep recursion) led to a redesign based on two properties of the problem rather than on a library function. All four objectives were met; the main new cost is the growing size of the numbers, which the predicted 'linear time' did not allow for.

Complete source code of the experiment

Task07/redesign.py

```
"""Q7 - Diagnose, redesign and validate: temple-step counting (program chosen from Q6)."""
import time, tracemalloc, json, sys

# ---------------- ORIGINAL (diagnosed program) ----------------
calls = 0
def ways_naive(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)

# ---------------- A. Diagnose: how often is each sub-problem solved? ----------------
from collections import Counter
seen = Counter()
def ways_traced(n):
    seen[n] += 1
    if n <= 1:
        return 1
    return ways_traced(n - 1) + ways_traced(n - 2)
ways_traced(5)
print("sub-problem repeat counts for n = 5:", dict(sorted(seen.items(), reverse=True)))
seen.clear(); ways_traced(25)
print("n = 25: total calls =", sum(seen.values()), "| distinct sub-problems =", len(seen),
      "| ways(1) alone solved", seen[1], "times | repeated work =", f"{1 - len(seen)/sum(seen.values()):.4%}")

# ---------------- C. Redesign: bottom-up with two rolling variables ----------------
def ways_dp(n):
    """Returns (value, additions). Keeps only ways(i-1) and ways(i)."""
    prev, cur, adds = 1, 1, 0               # ways(0), ways(1)
    for _ in range(n - 1):
        prev, cur = cur, prev + cur
        adds += 1
    return cur, adds

def ways_table(limit):
    """All values ways(1..limit) in one pass: what the trust actually prints."""
    out, prev, cur = [], 1, 1
    out.append(1)
    for _ in range(limit - 1):
        prev, cur = cur, prev + cur
        out.append(cur)
    return out

# ---------------- E. Validate: same inputs, counts and times ----------------
print("\n n | original calls | redesign additions | original time | redesign time | speed-up | equal")
rows = []
for n in (10, 20, 25, 30, 35):
    calls = 0
    t = time.perf_counter(); a = ways_naive(n); t_o = time.perf_counter() - t
    c_o = calls
    reps = 2000
    t = time.perf_counter()
    for _ in range(reps):
        b, adds = ways_dp(n)
    t_r = (time.perf_counter() - t) / reps
    rows.append((n, c_o, adds, t_o, t_r))
    print(f"{n:2} | {c_o:>14,} | {adds:>18} | {t_o:12.6f}s | {t_r*1e6:10.2f} us | {t_o/t_r:>8,.0f}x | {a == b}")
```

```
print("\nRecursion depth: original needs depth n; redesign needs none.")
print("recursion limit:", sys.getrecursionlimit())
try:
    ways_naive(1500)
except RecursionError:
    print("original ways_naive(1500) -> RecursionError (it would also need ~1e313 calls)")
v, adds = ways_dp(1000)
print(f"redesign ways_dp(1000): {adds} additions, {len(str(v))}-digit answer, no recursion limit involved")

# full table 1..1000 (the trust's requirement)
t = time.perf_counter(); tab = ways_table(1000); t_tab = time.perf_counter() - t
print(f"\nfull table ways(1..1000): {len(tab)} values in {t_tab*1e3:.3f} ms, last value has {len(str(tab[-1]))}
    digits")
# naive effort to build the same table (calls only, computed from the formula 2*ways(n)-1)
print("naive calls for table up to n=30 only:", f"{sum(2*x-1 for x in tab[:30]):,}", "(table to 1000 is
    impossible)")

# memory
tracemalloc.start(); ways_dp(1000); cur, peak_dp = tracemalloc.get_traced_memory(); tracemalloc.stop()
tracemalloc.start(); ways_table(1000); cur, peak_tab = tracemalloc.get_traced_memory(); tracemalloc.stop()
print(f"peak memory: rolling variables {peak_dp/1e3:.1f} KB, full table {peak_tab/1e3:.1f} KB")

# timing growth of the redesign
print("\nredesign time growth (average of 200 runs):")
g = []
for n in (100, 200, 400, 800, 1600):
    t = time.perf_counter()
    for _ in range(200): ways_dp(n)
    dt = (time.perf_counter() - t) / 200
    g.append((n, dt)); print(f"  n={n:5}: {dt*1e6:8.1f} us")

json.dump({"rows": rows, "growth": g}, open("Task07/data.json", "w"))
```